



Rapport de Projet

Projet de Réalité Virtuelle Adaptative

CogAdapt-VR

Formation Pilotée par l'Analyse Comportementale

Réalisé par :

- Khouloud BENNOUBY
- Asmaa TALAL
- Latifa AIT ALLA
- Badr KANDRI IDRISI
- Saad BENHAIMER
- Hamza ELMOURABIT

Encadré par :

- Pr. Hamid HRIMECH
- Madame Oumaima
DEROUECH

Table des matières

Remerciements	5
Introduction Générale	6
1 État de l'Art et Modélisation Théorique du DDA	7
1.1 Taxonomie des Systèmes DDA	7
1.1.1 Les Systèmes Experts Heuristiques	7
1.1.2 Les Systèmes Probabilistes Markoviens	7
1.1.3 L'Apprentissage Statistique Multivarié	7
1.2 Analyse des Signatures Comportementales en Réalité Virtuelle . . .	8
1.2.1 Les Signaux Physiologiques (EEG, ECG, GSR)	8
1.2.2 Les Signaux Cinématiques Haptiques (Tracking VR)	8
2 Conception de l'Environnement Immersif et Architecture Spatiale	9
2.1 Model du Système Cinétique : Le Tube Magique	9
2.2 Topologie et Spécifications des Scènes Applicatives	10
2.2.1 Scène Easy : Configuration Nominale et Monotonie Spatiale	11
2.2.2 Scène Hard : Complexité Cognitive et Permutation Spatiale	11
2.2.3 Scène Game Production : Intégration et Commutation Dynamique	11
3 Architecture Globale du Projet et Topologie Logicielle	12
3.1 Modélisation de la Topologie Logicielle Découplée	12
3.2 Pipeline d'Ingestion Temporelle et Acquisition Multi-Sources	12
3.3 Ingénierie Algorithmique des Features Cinématiques	14
3.3.1 Temps de Réaction	14
3.3.2 Vitesse Moyenne du Contrôleur Droit	14
3.3.3 Vitesse Maximale du Contrôleur Droit	14
3.3.4 Indice d'Hésitation Latérale (Axe X)	14

4	Technologies Utilisées et Environnement de Développement	15
4.1	Vue d'Ensemble de la Stack Technologique	15
4.2	Environnement de Développement Client VR	16
4.2.1	Unity 3D (Version 2022 LTS)	16
4.2.2	XR Interaction Toolkit (XRI)	16
4.2.3	C# et Visual Studio	16
4.3	Environnement Backend et Microservices	17
4.3.1	Python 3.10	17
4.3.2	Flask (Framework Web Micro-service)	17
4.3.3	MongoDB et MongoDB Compass	17
4.4	Environnement d'Intelligence Artificielle	18
4.4.1	Scikit-learn	18
4.4.2	Pandas et NumPy	18
4.4.3	Jupyter Notebook	19
4.4.4	Matplotlib et Seaborn	19
4.5	Outils de Versionnement et Collaboration	19
4.5.1	Git et GitHub	19
4.6	Synthèse de l'Environnement Technologique	20
5	Fondements Mathématiques et Algorithmiques du Modèle ML	22
5.1	Théorie des Forêts Aléatoires et Apprentissage d'Ensemble	22
5.2	La Fonction de Coût et Critère d'Impureté de Gini	23
5.2.1	Estimation de l'Erreur Out-Of-Bag (OOB)	24
5.3	Logique d'Annotation Multivariée pour l'Apprentissage	24
5.4	Validation Croisée Stratifiée et Équilibrage des Classes	24
5.5	Moteur d'Inférence Événementiel et Calibration des Seuils	25
6	Spécifications Techniques de l'Infrastructure et des APIs REST	26
6.1	Architecture et Protocoles de Communication	26
6.2	Modélisation des Échanges d'API (Payloads JSON)	27
6.2.1	Payload de Transmission d'Action (Unity -> Port 5000)	27
6.3	Code Source Complet du Composant d'Inférence Événementiel	28
6.3.1	Composant : brain.py	28
A	Scripts C# d'Acquisition et Moteurs Clients Unity	31
A.1	Script de Contrôle Global : GameManager.cs	31
A.2	Script d'Échantillonnage et de Télémétrie : DataCollector.cs	34
A.3	Générateur de Tâches Dynamiques : BallSpawnerHard.cs	37
A.4	Script d'Évaluation de Collision Spatiale : ZoneDetecteurHard.cs	39
	Conclusion Générale	41

Liste des tableaux

4.1	Répartition des technologies par couche fonctionnelle	15
4.2	Récapitulatif des technologies et versions utilisées dans CogAdapt-VR	21
6.1	Spécifications des points d'accès (Endpoints) de l'infrastructure logicielle	26

Table des figures

2.1	Scène Easy avec positions fixes des réceptacles	9
2.2	Scène Hard avec permutation dynamique des couleurs	10
2.3	Organisation hiérarchique des GameObjects dans Unity	10
3.1	Visualisation des collections sous MongoDB Compass	13
4.1	Cartographie de l'écosystème technologique de CogAdapt-VR	20
5.1	Architecture structurelle du classifieur Random Forest par vote d'arbres indépendants	22
6.1	Logs d'exécution du serveur d'inférence (brain.py) lors de la réception d'un bloc de comportement et calibration du seuil à 35%	27

Remerciements

Nous tenons tout d'abord à exprimer notre profonde gratitude envers l'ensemble du corps professoral et administratif de l'**École Nationale des Sciences Appliquées (ENSA)**, pour la qualité de la formation et l'accompagnement constant tout au long de notre parcours universitaire.

nos remerciements s'adressent tout particulièrement à notre encadrant pédagogique, **Madame Oumaima DEROUECH**, pour sa disponibilité, ses orientations précieuses et ses conseils avisés qui ont grandement contribué à la structuration et à la réussite de ce Projet de Fin de module.

Introduction Générale

L'avènement des technologies immersives tridimensionnelles, portées par la Réalité Virtuelle (VR), redéfinit fondamentalement l'interaction homme-machine. Contrairement aux interfaces bi-dimensionnelles conventionnelles, les environnements immersifs capturent la motricité humaine dans sa globalité. Cette dynamique génère d'immenses flux continus de coordonnées spatiales complexes. Ces données, riches en descripteurs latents, ouvrent une voie prometteuse pour concevoir des environnements interactifs adaptatifs intelligents, capables de se moduler à la volée selon le profil attentionnel et physique de l'utilisateur.

En psychologie cognitive, le maintien d'un niveau d'engagement idéal au cours d'une tâche interactive est modélisé par la théorie du Flow. Formulée par Mihaly Csikszentmihalyi, elle définit un état d'activation optimale où le niveau de défi d'une tâche coïncide idéalement avec les aptitudes du sujet. Si le défi est insuffisant, l'opérateur bascule vers une sous-charge cognitive induisant l'ennui. À l'inverse, si la complexité surclasse ses aptitudes cinématiques, une surcharge cognitive apparaît. Celle-ci génère du stress, de l'anxiété et une dégradation motrice marquée.

Pour contourner cette rigidité, les systèmes d'Ajustement Dynamique de la Difficulté (DDA – *Dynamic Difficulty Adjustment*) ont vu le jour. Historiquement, ces systèmes reposaient sur des modèles experts rigides ou des règles déterministes simplistes de type `if/else`. Ces approches échouent à formaliser la subtilité non linéaire des mouvements physiques humains et les dynamiques de fatigue.

Ce Projet de Fin de module s'inscrit en rupture avec ces architectures empiriques. Il propose la conception et l'implémentation d'un moteur de régulation cognitive autonome, guidé par l'IA pure et fondé sur l'apprentissage statistique supervisé. En exploitant la cinématique des capteurs VR, notre système capture, stocke et analyse des blocs de comportements via un classifieur de Forêt Aléatoire (*Random Forest*) interconnecté par une architecture distribuée de services REST. L'objectif applicatif est de valider la viabilité d'un modèle d'apprentissage pour réguler en continu, sans transition perceptive, l'expérience utilisateur.

État de l'Art et Modélisation Théorique du DDA

1.1 Taxonomie des Systèmes DDA

L'évolution des mécanismes de régulation de la difficulté dans les logiciels interactifs s'articule autour de trois générations d'architectures bien distinctes.

1.1.1 Les Systèmes Experts Heuristiques

Fondés sur des règles statiques édictées par des concepteurs métiers, ces systèmes évaluent des compteurs isolés, comme le nombre de réussites consécutives, pour appliquer des incréments fixes de complexité. L'inconvénient majeur réside dans leur incapacité à généraliser face à un utilisateur au comportement atypique.

1.1.2 Les Systèmes Probabilistes Markoviens

Exploitant les chaînes de Markov discrètes, ces modèles estiment la probabilité de transition d'un utilisateur vers un état d'échec ou d'abandon. Bien que plus dynamiques, ils manquent de finesse pour intégrer des variables physiques continues à haute fréquence et souffrent de l'explosion de l'espace des états.

1.1.3 L'Apprentissage Statistique Multivarié

Cette approche moderne, adoptée dans ce projet, fusionne des variables temporelles, de performance et cinématiques au sein d'un même espace de caractéristiques. Elle confie à un estimateur non linéaire la tâche de cartographier la charge cognitive de l'utilisateur, offrant ainsi une granularité d'adaptation fine et individualisée.

1.2 Analyse des Signatures Comportementales en Réalité Virtuelle

La capture des données en environnement virtuel se confronte à deux grandes catégories de variables biologiques et comportementales.

1.2.1 Les Signaux Physiologiques (EEG, ECG, GSR)

Ces signaux offrent une excellente précision sur l'état d'activation du système nerveux autonome (fréquence cardiaque, conductance cutanée). Cependant, ils s'avèrent hautement intrusifs pour l'utilisateur en situation de production réelle, nécessitent du matériel lourd et induisent un fort bruit de mesure lié aux mouvements musculaires.

1.2.2 Les Signaux Cinématiques Haptiques (Tracking VR)

Il s'agit des flux de positions et d'orientations nativement générés par les casques et contrôleurs VR. Notre approche privilégie cette catégorie de signaux : non intrusive, elle exploite les micro-variations motrices (vitesse, accélérations, hésitations directionnelles) comme des variables proxy de la charge mentale, capturant la fatigue attentionnelle directement à travers l'action de l'opérateur.

Le système d'interaction principal repose sur un protocole applicatif développé sous Unity 3D, exploitant le framework *XR Interaction Toolkit*. L'utilisateur évolue dans un espace tridimensionnel dont la tâche principale consiste à trier des projectiles physiques sphériques.

9

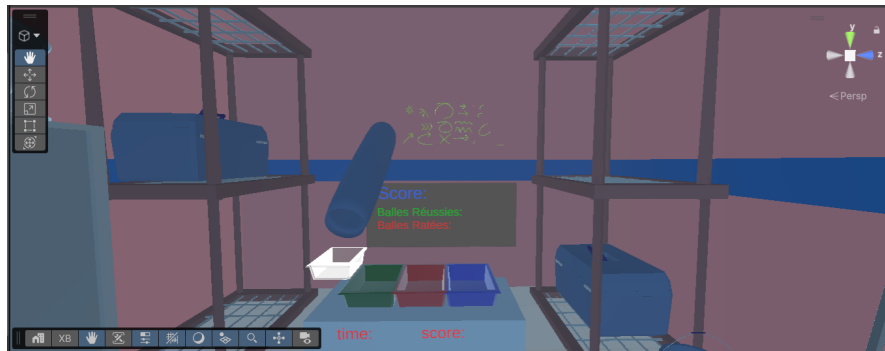


FIGURE 2.2 – Scène Hard avec permutation dynamique des couleurs

2.2 Topologie et Spécifications des Scènes Applicatives

Pour garantir la robustesse de l'analyse de données et dissocier les variables de confusion lors des phases d'expérimentation, l'architecture sous Unity a été compartimentée en trois scènes distinctes et étanches.

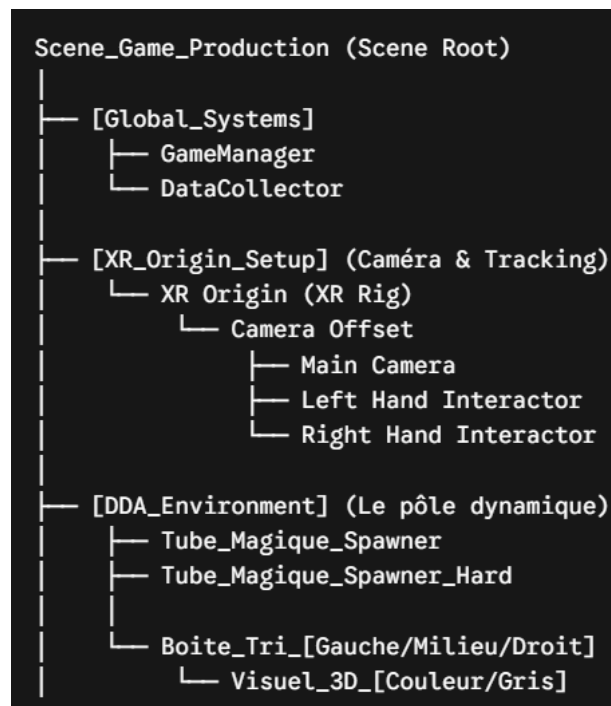


FIGURE 2.3 – Organisation hiérarchique des GameObjects dans Unity

2.2.1 Scène Easy : Configuration Nominale et Monotonie Spatiale

La scène *Easy* modélise un environnement à faible charge cognitive. La disposition géométrique des trois réceptacles de tri (boîtes) est fixe et prédictible dans l'espace euclidien. Les axes de correspondance entre l'identité visuelle du projectile et le conteneur physique cible sont univoques et invariables au cours du temps :

- **Conteneur Latéral Gauche** : Dédié exclusivement à la capture des projectiles de couleur Verte (C_V).
- **Conteneur Central** : Dédié exclusivement à la capture des projectiles de couleur Rouge (C_R).
- **Conteneur Latéral Droit** : Dédié exclusivement à la capture des projectiles de couleur Bleue (C_B).

2.2.2 Scène Hard : Complexité Cognitive et Permutation Spatiale

La scène *Hard* introduit une double rupture mécanique destinée à saturer les capacités cognitives de l'utilisateur et induire une variabilité cinématique prononcée :

1. **Permutation Aléatoire Continue** : À chaque nouvelle instanciation d'un projectile par le Tube Magique, l'ordonnancement logique des couleurs associées aux réceptacles physiques subit une permutation aléatoire. Si l'on note $S = \{C_R, C_V, C_B\}$ l'ensemble des couleurs, l'application linéaire d'attribution spatiale aux réceptacles $A : \{G, M, D\} \rightarrow S$ change à chaque itération via un algorithme de brassage aléatoire.
2. **Mécanique de Dissimulation Temporelle (Flash)** : Lors de l'apparition de la balle, les textures émissives des conteneurs affichent leurs couleurs respectives pendant une durée stricte notée $t_{\text{flash}} = 1.0$ seconde. À l'expiration de ce délai, une coroutine Unity force la substitution des matériaux vers un gris neutre identique (M_{neutral}).

2.2.3 Scène Game Production : Intégration et Commutation Dynamique

La scène de production constitue le livrable applicatif final. Elle s'initialise dans un état de difficulté de base (*Easy*), puis interroge de manière asynchrone le moteur d'IA. Dès que le serveur d'inférence notifie un changement d'état (code 1 ou 2), la scène applique dynamiquement les transformations géométriques et matérielles adéquates sans interrompre l'expérience de l'utilisateur.

Architecture Globale du Projet et Topologie Logicielle

3.1 Modélisation de la Topologie Logicielle Découplée

L'architecture de notre système repose sur un découpage fonctionnel strict en couches afin de dissocier le calcul graphique en temps réel (Unity), l'ingestion massive des flux de données (Serveur d'acquisition) et le calcul distribué des prédictions cognitives (Moteur DDA).

Cette topologie découplée garantit qu'aucune surcharge liée aux calculs matriciels du modèle d'apprentissage automatique n'impacte le taux de rafraîchissement d'images du casque de réalité virtuelle, écartant ainsi tout risque de cinétose (*motion sickness*) pour l'utilisateur.

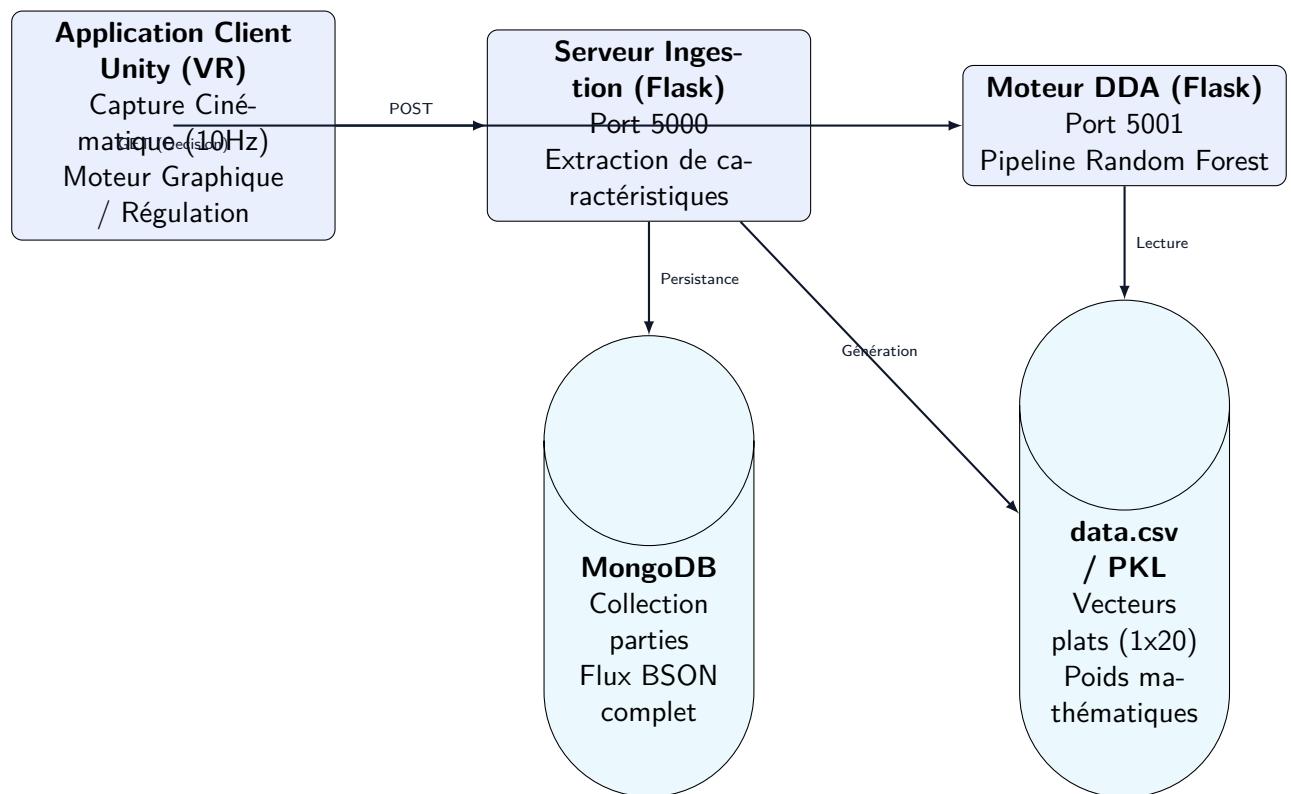
3.2 Pipeline d'Ingestion Temporelle et Acquisition Multi-Sources

L'acquisition des données cinématiques est orchestrée par le script `DataCollector.cs`. Le flux directionnel et positionnel de l'utilisateur est capturé au sein du repère cartésien de Unity à une fréquence fixe de 10 Hz ($\Delta t = 0.1$ s).

Chaque trame d'échantillonnage, modélisée par la structure `FrameTracking`, encapsule le vecteur temporel relatif ainsi que les triplets de coordonnées euclidiennes :

$$P(t) = \begin{bmatrix} x(t) & y(t) & z(t) \end{bmatrix}^T$$

Le pipeline de données implémente une stratégie de stockage hybride :



- **Couche d’Historisation NoSQL (MongoDB) :** Le flux complet de mouvements de chaque essai (`fluxMouvements`) est transmis via une requête HTTP POST au serveur d’ingestion (Port 5000) et inséré dans MongoDB au format BSON pour préserver l’historique massif du projet.

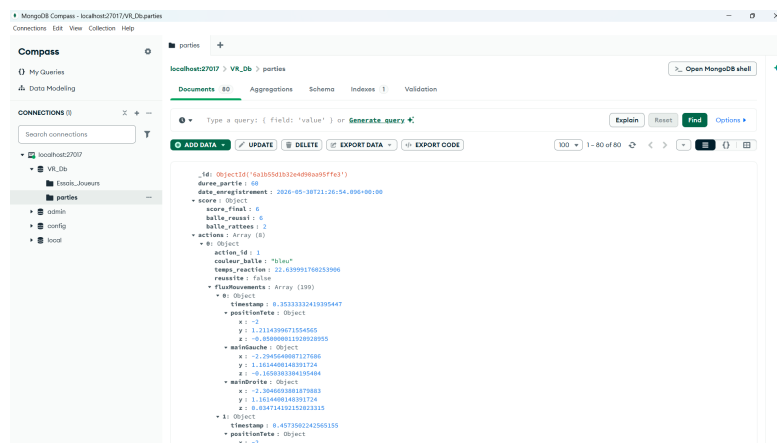


FIGURE 3.1 – Visualisation des collections sous MongoDB Compass

- **Couche d'Inférence Structurée (data.csv)** : Le serveur condense l'essai en un vecteur de caractéristiques cinématiques fondamentales directement écrit à la fin du fichier `data.csv`, permettant une lecture séquentielle rapide par le script d'inférence.

3.3 Ingénierie Algorithmique des Features Cinématiques

Pour transformer le signal brut discret en variables exploitables, quatre descripteurs majeurs sont calculés à la fin de chaque lancer :

3.3.1 Temps de Réaction

Mesure absolue de la latence cognitive de traitement visuel :

$$T_{\text{reaction}} = t_{\text{collision}} - t_{\text{apparition}}$$

3.3.2 Vitesse Moyenne du Contrôleur Droit

Intègre la composante dynamique de déplacement du bras de l'opérateur :

$$v_i = \frac{\sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2 + (z_i - z_{i-1})^2}}{t_i - t_{i-1}}$$

$$\mu_v = \frac{1}{N-1} \sum_{i=2}^N v_i$$

3.3.3 Vitesse Maximale du Contrôleur Droit

Capte l'énergie cinétique crête déployée lors du geste de lancer :

$$v_{\text{max}} = \max_{i \in [2, N]} (v_i)$$

3.3.4 Indice d'Hésitation Latérale (Axe X)

Quantifie l'indécision spatiale par l'écart-type de la position sur l'axe X :

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad \Rightarrow \quad \sigma_x = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Technologies Utilisées et Environnement de Développement

Ce chapitre présente l'ensemble de l'écosystème technologique mobilisé pour la conception, le développement et le déploiement du système **CogAdapt-VR**. Chaque outil a été sélectionné selon des critères précis d'interopérabilité, de performance et d'adéquation aux contraintes du temps réel immersif.

4.1 Vue d'Ensemble de la Stack Technologique

L'architecture du projet repose sur trois grandes couches fonctionnelles, chacune associée à un ensemble d'outils spécialisés :

Couche	Rôle	Technologies
Client VR	Rendu 3D, capture cinématique, interaction XR	Unity, C#, XR Toolkit
Backend & API	Ingestion REST, persistance, orchestration	Python, Flask, MongoDB
Intelligence	Entraînement, inférence, analyse de données	Scikit-learn, Pandas, Jupyter

TABLE 4.1 – Répartition des technologies par couche fonctionnelle

4.2 Environnement de Développement Client VR

4.2.1 Unity 3D (Version 2022 LTS)

Unity Technologies est le moteur de rendu temps réel au cœur de la couche client. Sa version LTS (Long-Term Support) 2022 garantit la stabilité nécessaire aux environnements de production VR. Unity orchestre le rendu stéréoscopique à haute fréquence, la simulation physique des objets rigides (**Rigidbody**), la gestion des scènes et l'exécution des scripts **C#** qui pilotent la logique applicative. Le pipeline de rendu **Universal Render Pipeline (URP)** a été retenu pour ses performances optimisées sur casques XR autonomes.

Fonctionnalités clés exploitées :

- Moteur physique intégré (**PhysX**)
- Coroutines asynchrones
- **UnityWebRequest**
- Gestion dynamique des scènes



4.2.2 XR Interaction Toolkit (XRI)

Le **XR Interaction Toolkit** est le framework officiel de Unity pour le développement d'expériences en Réalité Virtuelle et Augmentée. Il fournit une abstraction matérielle permettant de cibler simultanément différents casques (Meta Quest, HTC Vive, Valve Index) sans modification du code. Dans CogAdapt-VR, il gère la détection des interactions de saisie (**XRGrabInteractable**), la validation des poses et le tracking des contrôleurs six degrés de liberté (6-DoF).

4.2.3 C# et Visual Studio

L'ensemble de la logique applicative côté client est implémentée en **C#**, le langage natif de Unity. Son typage statique fort garantit la robustesse des structures de données cinématiques (**FrameTracking**, **ActionPayload**). **Visual Studio 2022** a été utilisé comme environnement de développement intégré (IDE), offrant l'autocomplétion Intellisense, le débogage pas-à-pas et l'intégration Git.



4.3 Environnement Backend et Microservices

4.3.1 Python 3.10

Python 3.10 constitue le socle de l'ensemble du backend. Son écosystème scientifique mature (NumPy, Pandas, Scikit-learn) et sa facilité d'intégration avec les frameworks web légers en font le choix naturel pour un moteur d'IA embarqué dans un service REST. La version 3.10 a notamment été retenue pour ses améliorations de performance sur les opérations d'entrées/sorties intensives et sa compatibilité avec les bibliothèques de machine learning utilisées.



4.3.2 Flask (Framework Web Micro-service)

Flask est le micro-framework web WSGI utilisé pour exposer les deux micro-services REST du système. Sa légèreté est primordiale : contrairement à Django ou FastAPI, Flask introduit une surcharge mémoire minimale, ce qui garantit une latence de réponse inférieure à 50 ms pour les requêtes d'inférence. Deux instances Flask indépendantes sont déployées : le **Serveur d'Acquisition** (port 5000) chargé de la réception et de la persistance des données, et le **Moteur DDA** (port 5001) hébergeant le pipeline de prédiction Random Forest.



4.3.3 MongoDB et MongoDB Compass

MongoDB est le système de gestion de bases de données NoSQL orienté documents retenu pour l'historisation des flux cinématiques bruts. Le format **BSON**

(Binary JSON) permet le stockage natif des tableaux de structures `FrameTracking` sans transformation préalable, préservant intégralement la hiérarchie des données de session. **MongoDB Compass** a servi d'interface graphique pour l'exploration et la validation des collections en cours de développement.



4.4 Environnement d'Intelligence Artificielle

4.4.1 Scikit-learn

Scikit-learn est la bibliothèque centrale du pipeline de machine learning. Elle fournit l'implémentation du classifieur `RandomForestClassifier` avec le critère d'impureté de Gini, les outils de validation croisée stratifiée (`StratifiedKFold`), les métriques d'évaluation (matrices de confusion, F1-score pondéré) et les utilitaires de sérialisation du modèle entraîné au format `.pkl` via `joblib`. Sa compatibilité native avec NumPy et Pandas garantit un pipeline de données sans copie en mémoire.



4.4.2 Pandas et NumPy

Pandas est utilisé pour toutes les opérations d'ingénierie de features : lecture séquentielle du fichier `data.csv`, filtrage, calcul des statistiques descriptives (moyennes, écarts-types des vitesses) et gestion des valeurs manquantes. **NumPy** intervient en sous-couche pour les calculs vectoriels et matriciels (distances euclidiennes, normalisations) sur les vecteurs de caractéristiques de dimension 20.



4.4.3 Jupyter Notebook

Jupyter Notebook a constitué l'environnement interactif d'expérimentation scientifique. Il a permis l'exploration itérative des données, la visualisation des distributions comportementales (histogrammes de temps de réaction, courbes de vitesse), le prototypage des hyperparamètres du Random Forest et la génération des rapports d'évaluation. Sa capacité à combiner code exécutable, formules LaTeX et visualisations graphiques en fait un outil de recherche reproductible adapté à la démarche expérimentale du projet.



4.4.4 Matplotlib et Seaborn

Matplotlib et **Seaborn** ont été mobilisés pour la visualisation analytique des données d'entraînement et des résultats de validation. Les matrices de confusion normalisées, les courbes d'importance des features et les distributions de classes ont été générées via ces bibliothèques, fournissant les supports graphiques nécessaires à l'analyse des performances du modèle.



4.5 Outils de Versionnement et Collaboration

4.5.1 Git et GitHub

Git a assuré le versionnement décentralisé de l'ensemble du code source du projet. La plateforme **GitHub** a hébergé le dépôt centralisé, facilitant la collaboration entre les membres de l'équipe via un workflow de branches fonctionnelles

(`feature/`, `main`). Les artefacts de modèle sérialisés (`.pkl`), les notebooks d'analyse et les scripts Unity ont été versionnés conjointement pour garantir la reproductibilité des expériences.



4.6 Synthèse de l'Environnement Technologique

La figure suivante récapitule l'ensemble des technologies déployées et leurs interactions au sein de l'écosystème CogAdapt-VR.

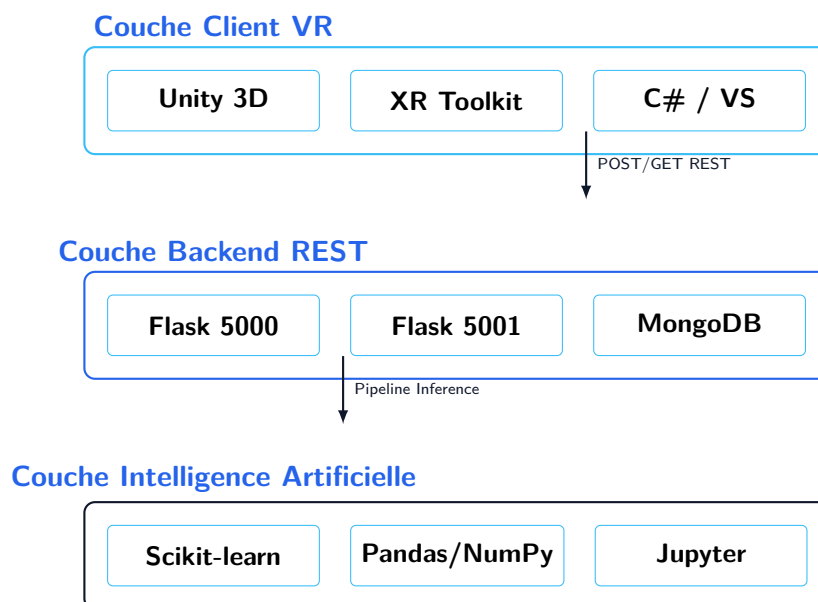


FIGURE 4.1 – Cartographie de l'écosystème technologique de CogAdapt-VR

Le tableau suivant présente une synthèse des versions et rôles de chaque outil :

Technologie	Catégorie	Version	Rôle dans CogAdapt-VR
Unity 3D	Moteur VR	2022 LTS	Rendu stéréoscopique, simulation physique
XR Interaction TK	Framework XR	2.x	Interaction 6-DoF, tracking contrôleurs
C# / Visual Studio	Langage / IDE	.NET 6	Logique client, collecte cinématique
Python	Langage backend	3.10	Orchestration backend et IA
Flask	Framework REST	2.3	Exposition des microservices API
MongoDB	Base NoSQL	6.0	Persistance flux BSON temps réel
Scikit-learn	Machine Learning	1.3	Random Forest, validation, métriques
Pandas	Data Engineering	2.0	Ingénierie de features, I/O CSV
NumPy	Calcul vectoriel	1.25	Opérations matricielles (features)
Jupyter Notebook	IDE Scientifique	7.x	Exploration et expérimentation ML
Matplotlib/Seaborn	Visualisation	3.8 / 0.13	Analyse statistique, courbes ROC
Git / GitHub	Versionnement	2.x	Collaboration et CI du code source

TABLE 4.2 – Récapitulatif des technologies et versions utilisées dans CogAdapt-VR

Fondements Mathématiques et Algorithmiques du Modèle ML

5.1 Théorie des Forêts Aléatoires et Apprentissage d'Ensemble

Le modèle de Machine Learning implémenté pour classifier l'état cognitif du joueur est une **Forêt Aléatoire** (*Random Forest Classifier*). Il s'agit d'un algorithme d'apprentissage supervisé basé sur le principe du *Machine Learning d'Ensemble*, combinant une multitude d'arbres de décision indépendants pour générer une prédiction stable et robuste.

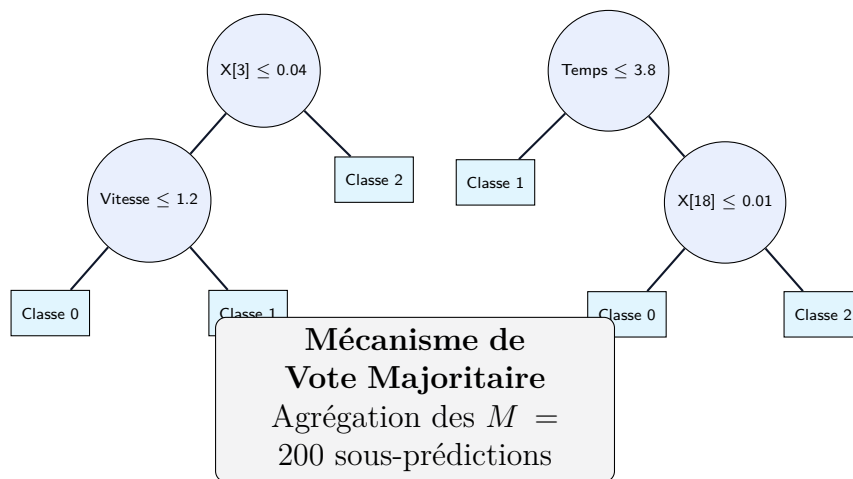


FIGURE 5.1 – Architecture structurelle du classifieur Random Forest par vote d'arbres indépendants

Le fonctionnement de la forêt s'appuie sur deux piliers mathématiques majeurs :

1. **Le Bagging (Bootstrap Aggregation)** : Chaque arbre de la forêt est entraîné sur un sous-ensemble de données distinct, généré par un échantillonnage aléatoire avec remplacement (*tirage bootstrap*) sur le dataset original. Si le dataset initial contient N lignes, chaque échantillon bootstrap possède également N lignes, mais contient environ 63.2% d'observations uniques, le reste étant des doublons.
2. **La Sélection Aléatoire de Caractéristiques (*Feature Subspace Sampling*)** : Lors de la division (*split*) de chaque nœud au sein d'un arbre, l'algorithme ne cherche pas la meilleure variable parmi l'intégralité des 20 caractéristiques disponibles. Il sélectionne aléatoirement un sous-ensemble de taille $m = \sqrt{20} \approx 4$ variables, puis détermine la coupure optimale uniquement parmi ces dernières.

5.2 La Fonction de Coût et Critère d'Impureté de Gini

Un arbre de décision ne possède pas de fonction de coût globale continue optimisable par descente de gradient (comme la régression logistique). Sa "fonction de coût" est une fonction de partitionnement récursive basée sur la minimisation d'un critère d'impureté. Dans notre modèle, nous exploitons l'**Impureté de Gini**.

Soit un nœud m représentant un sous-ensemble de données Q_m . On note p_{mk} la proportion d'instances appartenant à la classe $k \in \{0, 1, 2\}$ au sein de ce nœud :

$$p_{mk} = \frac{1}{|Q_m|} \sum_{i \in Q_m} I(y_i = f_k)$$

L'impureté de Gini au nœud m , notée $H(Q_m)$, mesure la probabilité d'une erreur de classification si une instance était étiquetée de manière aléatoire selon la distribution des classes du nœud :

$$H(Q_m) = \sum_{k=0}^2 p_{mk}(1 - p_{mk}) = 1 - \sum_{k=0}^2 p_{mk}^2$$

Lorsqu'une caractéristique j et un seuil t sont évalués pour diviser le nœud m en deux sous-nœuds, Gauche (Q_m^{left}) et Droit (Q_m^{right}), l'algorithme cherche à maximiser le gain d'information, ce qui revient à minimiser la fonction de coût locale de scission $G(j, t)$:

$$G(j, t) = \frac{|Q_m^{\text{left}}|}{|Q_m|} H(Q_m^{\text{left}}) + \frac{|Q_m^{\text{right}}|}{|Q_m|} H(Q_m^{\text{right}})$$

5.2.1 Estimation de l'Erreur Out-Of-Bag (OOB)

Un avantage fondamental du mécanisme de bootstrap réside dans l'évaluation interne non biaisée du modèle sans sous-ensemble de validation externe. Pour chaque observation x_i , l'erreur Out-Of-Bag est calculée en agrégeant uniquement les prédictions des arbres T_b pour lesquels x_i n'a pas servi lors de la phase d'apprentissage :

$$\hat{y}_{\text{OOB}}(x_i) = \arg \max_k \sum_{b: x_i \notin D_b} I(T_b(x_i) = k)$$

5.3 Logique d'Annotation Multivariée pour l'Apprentissage

L'étiquetage des données historiques s'affranchit des règles binaires simplistes pour adopter un modèle probabiliste croisé reflétant le comportement cinématique global de l'utilisateur au sein d'une fenêtre temporelle de 5 lancers :

$$\text{Label} = \begin{cases} 1 \text{ (Ennui)} & \text{si } \sum \text{reussite} \geq 4 \text{ et } (\bar{T}_{\text{reaction}} \leq 3.8 \text{ s ou } (\bar{v}_{\text{max}} \geq 1.2 \text{ m/s et } \bar{\sigma}_x \leq 0.015)) \\ 2 \text{ (Stress)} & \text{si } \sum \text{reussite} \leq 2 \text{ ou } \bar{T}_{\text{reaction}} \geq 5.0 \text{ s ou } \bar{\sigma}_x \geq 0.04 \\ 0 \text{ (Stable)} & \text{sinon} \end{cases} \quad (5.1)$$

5.4 Validation Croisée Stratifiée et Équilibrage des Classes

Pour contrer le fort déséquilibre de notre dataset original ($\mathcal{D} = \{0 : 208, 1 : 189, 2 : 613\}$), le critère d'impureté de Gini est ajusté mathématiquement via le paramètre `class_weight='balanced'`. Une pondération w_k est appliquée à la proportion de chaque classe lors du calcul de p_{mk} :

$$w_k = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_{\text{samples},k}}$$

L'évaluation utilise un algorithme de type `StratifiedGroupKFold` avec $K = 5$ partitions, évitant le *data leakage* en regroupant les fenêtres d'une même session au sein d'un seul bloc. Les performances validées atteignent une exactitude globale de 69.97% et un score F1 Macro de 0.6412.

5.5 Moteur d'Inférence Événementiel et Calibration des Seuils

En production, le serveur `brain.py` applique une logique événementielle stricte par paquets de 5 lancers accumulés (`to_decide == 5`). Le vecteur de caractéristiques de taille 20 est extrait.

Pour contrer le biais de la classe majoritaire, un ajustement des seuils décisionnels (**Threshold Tuning**) est déployé. Au lieu d'appliquer une fonction `argmax`, le système évalue les probabilités d'adaptation par rapport à un seuil critique de 35% :

```
1 probabilites = predictor._pipeline.predict_proba(input_vector)[0]
2 seuil_adaptation = 0.35
3
4 if len(probabilites) > 1 and probabilites[1] >= seuil_adaptation:
5     prediction_class = 1 # Priorite structurelle a l'Ennui ->
6     Mode HARD
7 elif len(probabilites) > 2 and probabilites[2] >= seuil_adaptation:
8     :
9     prediction_class = 2 # Priorite structurelle au Stress ->
10    Mode EASY
11 else:
12    prediction_class = 0 # Maintien de la Zone Stable de Flow
```

Une fois l'inférence validée, le cache du gestionnaire de session est intégralement vidé (`clear_session`), assurant l'indépendance statistique totale entre les cycles d'adaptation.

Spécifications Techniques de l'Infrastructure et des APIs REST

6.1 Architecture et Protocoles de Communication

L'échange d'informations inter-modules est géré par une architecture distribuée s'appuyant sur deux micro-services développés sous le framework Web Flask :

TABLE 6.1 – Spécifications des points d'accès (Endpoints) de l'infrastructure logicielle

Service	Endpoint	Méthode	Description
Données (5000)	/api/partie/demarrer	POST	Initialise l'instance de partie dans MongoDB.
Données (5000)	/api/partie/ajouter_action	POST	Enregistre les données cinématiques et synchronise le fichier CSV.
Données (5000)	/api/partie/terminer	POST	Clôture la partie et archive les scores finaux.
IA DDA (5001)	/api/dda/decision	GET	Déclenche l'inférence immédiate sur le paquet de cinq lancers.
IA DDA (5001)	/api/dda/reset	POST	Purge la mémoire et réinitialise le compteur <code>to_decide</code> .

```

2026-05-31 16:10:30,679 - INFO - Serveur de decision en temps reel
operationnel sur le PORT 5001
* Serving Flask app 'brain'
* Debug mode: off
2026-05-31 16:10:32,278 - INFO - WARNING: This is a development ser
ver. Do not use it in a production deployment. Use a production WSG
I server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5001
* Running on http://192.168.11.120:5001
2026-05-31 16:10:32,278 - INFO - Press CTRL+C to quit
2026-05-31 16:12:19,835 - INFO - 127.0.0.1 - - [31/May/2026 16:12:1
9] "POST /api/dda/reset HTTP/1.1" 200 -
2026-05-31 16:12:56,922 - INFO - 127.0.0.1 - - [31/May/2026 16:12:5
6] "GET /api/dda/decision HTTP/1.1" 200 -
2026-05-31 16:12:59,446 - INFO - 127.0.0.1 - - [31/May/2026 16:12:5
9] "GET /api/dda/decision HTTP/1.1" 200 -
2026-05-31 16:13:02,098 - INFO - 127.0.0.1 - - [31/May/2026 16:13:0
2] "GET /api/dda/decision HTTP/1.1" 200 -
2026-05-31 16:13:04,400 - INFO - 127.0.0.1 - - [31/May/2026 16:13:0
4] "GET /api/dda/decision HTTP/1.1" 200 -
|

```

FIGURE 6.1 – Logs d'exécution du serveur d'inférence (brain.py) lors de la réception d'un bloc de comportement et calibration du seuil à 35%

6.2 Modélisation des Échanges d'API (Payloads JSON)

6.2.1 Payload de Transmission d'Action (Unity -> Port 5000)

```

1 {
2   "partie_id": "6659c2b4f1a2c3456789abcd",
3   "action_id": 14,
4   "couleur_balle": "Rouge",
5   "temps_reaction": 3.425,
6   "reussite": true,
7   "fluxMouvements": [
8     {
9       "timestamp": 0.10,
10      "positionTete": {"x": 0.01, "y": 1.75, "z": -0.02},
11      "mainGauche": {"x": -0.25, "y": 1.10, "z": 0.35},
12      "mainDroite": {"x": 0.30, "y": 1.15, "z": 0.40}
13    }
14  ]
15 }

```

6.3 Code Source Complet du Composant d'Inférence Événementiel

L'implémentation industrielle finale et nettoyée du serveur de prédiction s'établit comme suit :

6.3.1 Composant : brain.py

```
1 from flask import Flask, jsonify, request
2 import pandas as pd
3 import os
4 import logging
5 import numpy as np
6
7 from inference import DDAPredictor, LancerFeatures
8 from session_manager import InMemorySessionManager
9
10 logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(
11    levelname)s - %(message)s')
12 logger = logging.getLogger(__name__)
13
14 app = Flask(__name__)
15 nom_fichier_csv = "data.csv"
16 path_modele_pkl = "preprocessors.pkl"
17 PARTIE_SESSION_ID = "partie_vr_dda_live"
18
19 session_manager = InMemorySessionManager()
20 predictor = DDAPredictor(model_path=path_modele_pkl,
21     session_manager=session_manager)
22
23 decision_actuelle = 0
24 dernier_action_id_traite = 0
25 to_decide = 0
26 lancers_du_paquet_courant = []
27
28 def executer_inference_sur_paquet_strict():
29     global decision_actuelle, dernier_action_id_traite, to_decide,
30     lancers_du_paquet_courant
31     if not os.path.exists(nom_fichier_csv) or os.path.getsize(
32         nom_fichier_csv) == 0:
33         return decision_actuelle
34     try:
35         df = pd.read_csv(nom_fichier_csv)
36         if len(df) == 0: return decision_actuelle
37         dernier_lancer_df = df.iloc[-1]
38         action_id_actuel = int(dernier_lancer_df['action_id'])
```

```

35         if action_id_actuel == dernier_action_id_traite: return
decision_actuelle
36
37         dernier_action_id_traite = action_id_actuel
38         to_decide += 1
39         features_list = [
40             float(dernier_lancer_df['temps_reaction']),
41             float(dernier_lancer_df['vitesse_moyenne_main_droite'
]),
42             float(dernier_lancer_df['vitesse_max_main_droite']),
43             float(dernier_lancer_df['hesitation_axe_x'])
44         ]
45         lancers_du_paquet_courant.append(features_list)
46         if to_decide < 5:
47             decision_actuelle = 0
48             return decision_actuelle
49
50         input_vector = np.array(lancers_du_paquet_courant).flatten
().reshape(1, -1)
51         probabilites = predictor._pipeline.predict_proba(
input_vector)[0]
52         seuil_adaptation = 0.35
53
54         if len(probabilites) > 1 and probabilites[1] >=
seuil_adaptation:
55             prediction_class = 1
56         elif len(probabilites) > 2 and probabilites[2] >=
seuil_adaptation:
57             prediction_class = 2
58         else:
59             prediction_class = 0
60
61         decision_actuelle = prediction_class
62         to_decide = 0
63         lancers_du_paquet_courant = []
64         session_manager.clear_session(PARTIE_SESSION_ID)
65     except Exception as e:
66         logger.error(f"Erreur d'inference : {str(e)}")
67         to_decide = 0
68         lancers_du_paquet_courant = []
69     return decision_actuelle
70
71 @app.route('/api/dda/decision', methods=['GET'])
72 def obtenir_decision():
73     code_calcule = executer_inference_sur_paquet_strict()
74     status_text = "Stable"
75     if code_calcule == 1: status_text = "Ennuie"
76     elif code_calcule == 2: status_text = "Stress"

```

```
77     return jsonify({"prediction": code_calcule, "status":
78         status_text}), 200
79
80 @app.route('/api/dda/reset', methods=['POST'])
81 def r_initialiser_ia_partie():
82     global decision_actuelle, dernier_action_id_traite, to_decide,
83         lanciers_du_paquet_courant
84     session_manager.clear_session(PARTIE_SESSION_ID)
85     decision_actuelle, dernier_action_id_traite, to_decide,
86     lanciers_du_paquet_courant = 0, 0, 0, []
87     return jsonify({"status": "reset"}), 200
88
89 if __name__ == '__main__':
90     app.run(port=5001, debug=False, host='0.0.0.0')
```

Scripts C# d'Acquisition et Moteurs Clients Unity

Cette section regroupe l'intégralité des scripts C# constituant le cœur applicatif côté client.

A.1 Script de Contrôle Global : GameManager.cs

Le script suivant supervise l'état fonctionnel de la scène, gère le compte à rebours absolu et invoque les fonctions de ré-ordonnancement spatial.

```
1 using UnityEngine;
2 using TMPro;
3 using System.Collections;
4 using UnityEngine.Networking;
5
6 public class GameManager : MonoBehaviour
7 {
8     public static GameManager Instance;
9     public float tempsTotalConfigure = 120f;
10    private float tempsRestant;
11    private bool jeuEnCours = true;
12
13    public TextMeshProUGUI texteChrono;
14    public TextMeshProUGUI texteScore;
15    public GameObject panneauFinDeJeu;
16    public TextMeshProUGUI texteBallesReussies;
17    public TextMeshProUGUI texteBallesRatees;
18    public TextMeshProUGUI texteScoreFinal;
19
20    public GameObject leftController;
21    public GameObject rightController;
22    [HideInInspector] public string modeActuel = "Easy";
```



```
23
24     private int ballesReussies = 0;
25     private int ballesRatees = 0;
26     private float tempsAuDemarrage;
27
28     void Awake()
29     {
30         if (Instance == null) Instance = this;
31         tempsRestant = tempsTotalConfigure;
32     }
33
34     void Start()
35     {
36         if (panneauFinDeJeu != null) panneauFinDeJeu.SetActive(
37 false);
38         tempsAuDemarrage = Time.time;
39         MettreAJourUI();
40         StartCoroutine(ResetDDAServerCache());
41
42         BallSpawner spawnerEasy = Object.FindAnyObjectByType<
43 BallSpawner>();
44         if (spawnerEasy != null) spawnerEasy.ResetBall();
45     }
46
47     void Update()
48     {
49         if (!jeuEnCours) return;
50         if (tempsRestant > 0) {
51             tempsRestant -= Time.deltaTime;
52             MettreAJourUI();
53         } else {
54             tempsRestant = 0;
55             FinDuJeu();
56         }
57     }
58
59     void MettreAJourUI()
60     {
61         int minutes = Mathf.FloorToInt(tempsRestant / 60);
62         int secondes = Mathf.FloorToInt(tempsRestant % 60);
63         if (texteChrono != null) texteChrono.text = string.Format(
64 "Temps: {0:00}:{1:00}", minutes, secondes);
65         if (texteScore != null) texteScore.text = "Score: " +
66 ballesReussies;
67     }
68
69     public void AjouterPoint() { if (jeuEnCours) { ballesReussies
70 ++; MettreAJourUI(); } }
71
72     public void BalleRatee() { if (jeuEnCours) ballesRatees++; }
```

```
67
68     public void ChangerDifficulteTempsReel(bool versHard)
69     {
70         string nouveauMode = versHard ? "Hard" : "Easy";
71         if (modeActuel == nouveauMode) return;
72         modeActuel = nouveauMode;
73
74         BallSpawner spawnerEasy = Object.FindAnyObjectByType<
BallSpawner>();
75         BallSpawnerHard spawnerHard = Object.FindAnyObjectByType<
BallSpawnerHard>();
76
77         GameObject[] tousLesObjets = GameObject.FindObjectsByType<
GameObject>(FindObjectsSortMode.None);
78         foreach (GameObject obj in tousLesObjets) {
79             if (obj.name.Contains("Balle")) Destroy(obj);
80         }
81
82         if (modeActuel == "Hard" && spawnerHard != null)
spawnerHard.ResetBall();
83         else if (modeActuel == "Easy" && spawnerEasy != null)
spawnerEasy.ResetBall();
84     }
85
86     void FinDuJeu()
87     {
88         jeuEnCours = false;
89         StartCoroutine(ResetDDAServerCache());
90         DataCollector dc = Object.FindAnyObjectByType<
DataCollector>();
91         if (dc != null) dc.CloturerPartie(tempsTotalConfigure,
ballesReussies, ballesRatees);
92         if (panneauFinDeJeu != null) panneauFinDeJeu.SetActive(
true);
93     }
94
95     IEnumerator ResetDDAServerCache()
96     {
97         string url = "http://127.0.0.1:5001/api/dda/reset";
98         using (UnityWebRequest request = UnityWebRequest.
PostWwwForm(url, "")) {
99             yield return request.SendWebRequest();
100         }
101     }
102 }
```

A.2 Script d'Échantillonnage et de Télémétrie : DataCollector.cs

Ce module abstrait capture les variables spatiales à 10Hz et orchestre l'émission asynchrone des trames vers les points de terminaison HTTP de l'API.

```

1 using UnityEngine;
2 using System.Collections.Generic;
3 using UnityEngine.Networking;
4 using System.Collections;
5
6 public class DataCollector : MonoBehaviour
7 {
8     public Transform head; public Transform leftHand; public
    Transform rightHand;
9     private string apiBaseURL = "http://localhost:5000/api/partie"
    ;
10    private string apiDdaURL = "http://localhost:5001/api/dda/
    decision";
11    private string partieId = ""; private int compteurActionId =
    0;
12
13    [System.Serializable]
14    public struct Vector3Data { public float x; public float y;
    public float z; }
15    [System.Serializable]
16    public struct FrameTracking { public float timestamp; public
    Vector3Data positionTete; public Vector3Data mainGauche; public
    Vector3Data mainDroite; }
17
18    [System.Serializable]
19    public class ActionPayload { public string partie_id; public
    int action_id; public string couleur_balle; public float
    temps_reaction; public bool reussite; public List<FrameTracking
    > fluxMouvements; }
20
21    [System.Serializable]
22    public class DDAPredictionResponse { public int prediction;
    public string status; }
23
24    private List<FrameTracking> donneesMouvementActuel = new List<
    FrameTracking>();
25    private bool estEnTrainDeTracker = false; private float
    chronoDebutEssai; private float decompTEmps = 0f;
26
27    void Start() { StartCoroutine(InitialiserPartieSurServeur());
    }
28    void Update() { if (estEnTrainDeTracker)

```

```

EnregistrerFrameFrequentielle(); }
29
30 public void DemarrerTracking() { donneesMouvementActuel.Clear()
; chronoDebutEssai = Time.time; estEnTrainDeTracker = true; }
31
32 private void EnregistrerFrameFrequentielle()
33 {
34     decompTEmps += Time.deltaTime;
35     if (decompTEmps >= 0.1f) {
36         decompTEmps = 0f;
37         FrameTracking frame = new FrameTracking {
38             timestamp = Time.time - chronoDebutEssai,
39             positionTete = FormaterVector3(head),
40             mainGauche = FormaterVector3(leftHand),
41             mainDroite = FormaterVector3(rightHand)
42         };
43         donneesMouvementActuel.Add(frame);
44     }
45 }
46
47 private Vector3Data FormaterVector3(Transform t)
48 {
49     if (t == null) return new Vector3Data { x = 0, y = 0, z =
0 };
50     return new Vector3Data { x = t.position.x, y = t.position.
y, z = t.position.z };
51 }
52
53 public void EnregistrerEssai(string couleur, float temps, bool
estReussi)
54 {
55     estEnTrainDeTracker = false; compteurActionId++;
56     ActionPayload payload = new ActionPayload {
57         partie_id = partieId, action_id = compteurActionId,
couleur_balle = couleur, temps_reaction = temps, reussite =
estReussi, fluxMouvements = new List<FrameTracking>(
donneesMouvementActuel)
58     };
59     StartCoroutine(EnvoyerRequetePOST(apiBaseUrl + "/"
ajouter_action", JsonUtility.ToJson(payload)));
60 }
61
62 private IEnumerator InitialiserPartieSurServeur()
63 {
64     using (UnityWebRequest req = new UnityWebRequest(
apiBaseUrl + "/demarrer", "POST")) {
65         req.downloadHandler = new DownloadHandlerBuffer();
66         req.SetRequestHeader("Content-Type", "application/json
");

```

```

67         yield return req.SendWebRequest();
68     }
69 }
70
71 private IEnumerator EnvoyerRequetePOST(string url, string json
72 )
73 {
74     using (UnityWebRequest request = new UnityWebRequest(url,
75 "POST")) {
76         byte[] bodyRaw = System.Text.Encoding.UTF8.GetBytes(
77 json);
78         request.uploadHandler = new UploadHandlerRaw(bodyRaw);
79         request.downloadHandler = new DownloadHandlerBuffer();
80         request.SetRequestHeader("Content-Type", "application/
81 json");
82         yield return request.SendWebRequest();
83         if (url.Contains("/ajouter_action")) StartCoroutine(
84 InterrogerMoteurDecisionDDA());
85     }
86 }
87
88 private IEnumerator InterrogerMoteurDecisionDDA()
89 {
90     using (UnityWebRequest request = UnityWebRequest.Get(
91 apiDdaURL)) {
92         yield return request.SendWebRequest();
93         if (request.result == UnityWebRequest.Result.Success)
94 {
95             DDAPredictionResponse res = JsonUtility.FromJson<
96 DDAPredictionResponse>(request.downloadHandler.text);
97             if (res.prediction == 1) GameManager.Instance.
98 ChangerDifficulteTempsReel(true);
99             else if (res.prediction == 2) GameManager.Instance
100 .ChangerDifficulteTempsReel(false);
101         }
102     }
103 }
104 }

```

A.3 Générateur de Tâches Dynamiques : BallSpawnerHard.cs

Ce script prend en charge l'instanciation physique des cibles tridimensionnelles et applique la logique de permutation.

```

1 using UnityEngine;
2 using System.Collections;
3 using System.Collections.Generic;
4
5 public class BallSpawnerHard : MonoBehaviour
6 {
7     public GameObject ballPrefab;
8     public float dureeFlashCouleurs = 1.0f;
9     private Vector3 startPosition;
10    [HideInInspector] public GameObject ball;
11    [HideInInspector] public float tempsApparitionBalle;
12
13    public Material matRouge; public Material matVert; public
14    Material matBleu; public Material matGrisNeutre;
15    public ZoneDetecteurHard detectionGauche; public
16    ZoneDetecteurHard detectionMilieu; public ZoneDetecteurHard
17    detectionDroite;
18    public GameObject visuelGauche; public GameObject visuelMilieu
19    ; public GameObject visuelDroite;
20
21    private Renderer renderGauche; private Renderer
22    renderMilieu; private Renderer renderDroite; private
23    Renderer ballRenderer;
24
25    void Awake() { startPosition = transform.position;
26    InitialiserRenderers(); }
27
28    private void InitialiserRenderers()
29    {
30        if (visuelGauche != null) renderGauche = visuelGauche.
31        GetComponent<Renderer>();
32        if (visuelMilieu != null) renderMilieu = visuelMilieu.
33        GetComponent<Renderer>();
34        if (visuelDroite != null) renderDroite = visuelDroite.
35        GetComponent<Renderer>();
36    }
37
38    public void SpawnNouvelleBalle()
39    {
40        if (GameManager.Instance != null && GameManager.Instance.
41        modeActuel != "Hard") return;
42        ball = Instantiate(ballPrefab, startPosition, Quaternion.

```

```

identity);
32     ballRenderer = ball.GetComponentInChildren<Renderer>();
33     Rigidbody rb = ball.GetComponent<Rigidbody>();
34     if (rb != null) rb.isKinematic = true;
35
36     ChangerCouleurBalleAleatoire();
37     StartCoroutine(LibererBalleApresStabilisation(rb));
38     StartCoroutine(FlashCouleursBoites());
39     Object.FindAnyObjectByType<DataCollector>().
DemarrerTracking();
40     tempsApparitionBalle = Time.time;
41 }
42
43 void ChangerCouleurBalleAleatoire()
44 {
45     int choix = Random.Range(0, 3);
46     if (choix == 0) { ball.name = "Balle_Rouge"; ballRenderer.
material = matRouge; }
47     else if (choix == 1) { ball.name = "Balle_Vert";
ballRenderer.material = matVert; }
48     else { ball.name = "Balle_Bleu"; ballRenderer.material =
matBleu; }
49 }
50
51 IEnumerator LibérerBalleApresStabilisation(Rigidbody rb)
52 {
53     yield return new WaitForSeconds(0.05f);
54     if (rb != null) { rb.isKinematic = false; rb.WakeUp(); }
55 }
56
57 IEnumerator FlashCouleursBoites()
58 {
59     List<string> couleurs = new List<string> { "Rouge", "Vert"
, "Bleu" };
60     for (int i = couleurs.Count - 1; i > 0; i--) {
61         int r = Random.Range(0, i + 1);
62         string tmp = couleurs[i]; couleurs[i] = couleurs[r];
couleurs[r] = tmp;
63     }
64     detectionGauche.couleurActuelle = couleurs[0];
65     detectionMilieu.couleurActuelle = couleurs[1];
66     detectionDroite.couleurActuelle = couleurs[2];
67
68     AppliquerMateriauBoite(rendererGauche, couleurs[0]);
69     AppliquerMateriauBoite(rendererMilieu, couleurs[1]);
70     AppliquerMateriauBoite(rendererDroite, couleurs[2]);
71
72     yield return new WaitForSeconds(dureeFlashCouleurs);
73     rendererGauche.material = matGrisNeutre; rendererMilieu.

```

```

material = matGrisNeutre; rendererDroite.material =
matGrisNeutre;
74 }
75
76 void AppliquerMateriauBoite(Renderer rend, string couleur)
77 {
78     if (rend == null) return;
79     if (couleur == "Rouge") rend.material = matRouge;
80     else if (couleur == "Vert") rend.material = matVert;
81     else if (couleur == "Bleu") rend.material = matBleu;
82 }
83
84 public void ResetBall() { StopAllCoroutines(); if (ball !=
null) Destroy(ball); SpawnNouvelleBalle(); }
85 }

```

A.4 Script d'Évaluation de Collision Spatiale : ZoneDetecteurHard.cs

Gère la validation cinématique, calcule le delta-temps par rapport à l'apparition et transmet les métriques au conteneur de télémétrie.

```

1 using UnityEngine;
2
3 public class ZoneDetecteurHard : MonoBehaviour
4 {
5     private BallSpawnerHard ballSpawner;
6     private DataCollector dataCollector;
7     public bool estLeSol = false;
8     [HideInInspector] public string couleurActuelle = "";
9
10    void Start()
11    {
12        ballSpawner = Object.FindFirstObjectByType<BallSpawnerHard
>();
13        dataCollector = Object.FindFirstObjectByType<DataCollector
>();
14    }
15
16    private void OnTriggerEnter(Collider other)
17    {
18        if (GameManager.Instance != null && GameManager.Instance.
modeActuel != "Hard") return;
19        if (other.CompareTag("Balle")) {
20            var interactable = other.GetComponent<UnityEngine.XR.
Interaction.Toolkit.Interactables.XRGrabInteractable>();

```



```
21         if (interactable != null && interactable.isSelected)
22             return;
23         string nomBalle = other.gameObject.name;
24         string couleurBallePure = nomBalle.Replace("Balle_", "
25 ").Replace("(Clone)", "").Trim().ToLower();
26         bool estCorrect = false; float tempsEcoule = Time.time
27         - ballSpawner.tempsApparitionBalle;
28
29         if (estLeSol) {
30             GameManager.Instance.BalleRatee();
31             dataCollector.EnregistrerEssai("Sol_" +
32 couleurBallePure, tempsEcoule, false);
33         } else {
34             string attendueMinuscule = couleurActuelle.ToLower
35 ();
36             if (couleurBallePure == attendueMinuscule &&
37 attendueMinuscule != "") estCorrect = true;
38
39             if (estCorrect) GameManager.Instance.AjouterPoint
40 ();
41             else GameManager.Instance.BalleRatee();
42
43             dataCollector.EnregistrerEssai(couleurBallePure,
44 tempsEcoule, estCorrect);
45         }
46         ballSpawner.ResetBall();
47     }
48 }
```

Conclusion Générale

Ce Projet de Fin de module a permis de concrétiser le déploiement d'une architecture d'Intelligence Artificielle pure dédiée à la régulation de la difficulté applicative au sein d'un écosystème de Réalité Virtuelle. En s'appuyant sur l'extraction rigoureuse de descripteurs cinématiques échantillonnés à 10 Hz, le système s'est affranchi des structures déterministes rigides traditionnelles pour adopter une approche statistique prédictive multivariée.

Les résultats obtenus lors de la validation croisée stratifiée par groupe démontrent la viabilité d'un modèle de **Forêt Aléatoire** fondé sur le critère d'impureté de Gini pour capturer l'état attentionnel et moteur du sujet. Enfin, l'introduction du réglage fin des seuils d'inférence à 35% offre une réponse efficace au problème majeur du biais de classe majoritaire nominale, assurant une bascule de difficulté réactive et fluide.

Ce travail pose des fondations applicatives solides pour l'intégration des méthodologies Big Data au service de l'expérience utilisateur adaptative de demain.